

User manual - Llama Desktop Launcher

Version 0.4.1 · English

1. What the application does

Llama Desktop Launcher installs or locates the `llama.cpp` engine, finds local GGUF models, and opens the `llama-server` interface in your browser. You can select local-only, LAN, or Tailscale access. Network access requires an API key. Traffic to other computers travels over the selected network.

The app installer **does not include models**. The official engine is downloaded on first use, or you can point the app to an existing installation. The tested engine version is `b11193`.

The interface has three screens: **Llama** for chat and starting the server, **Models** for finding GGUF files and viewing recommendations, and **Settings** for preparing the engine, folders, and profiles. Settings remains available in the navigation on every screen.

2. Requirements

Platform / Distribution	Architecture	Official package / Direct download	GPU backend
Windows (10 / 11)	x64	`Llama Desktop Launcher Setup 0.4.1.exe`	Vulkan, NVIDIA CUDA, AMD ROCm, Intel SYCL, or CPU
Linux (Debian, Ubuntu, Mint, Pop!_OS)	x64 (amd64)	`llama-desktop-launcher_0.4.1_amd64.deb`	Vulkan, CUDA, or CPU; ROCm/SYCL on x64
Linux (Debian, Ubuntu)	ARM64 (aarch64)	`llama-desktop-launcher_0.4.1_arm64.deb`	Vulkan or CPU
Linux (Fedora, RHEL, openSUSE)	x64 (x86_64)	`llama-desktop-launcher-0.4.1.x86_64.rpm`	Vulkan, CUDA, or CPU; ROCm/SYCL on x64
Linux (Fedora, RHEL, openSUSE)	ARM64 (aarch64)	`llama-desktop-launcher-0.4.1.aarch64.rpm`	Vulkan or CPU
Linux (Portable / All distros)	x64	`Llama Desktop Launcher-0.4.1.AppImage`	Vulkan, CUDA, or CPU; ROCm/SYCL on x64
Linux (Portable / All distros)	ARM64	`Llama Desktop Launcher-0.4.1-arm64.AppImage`	Vulkan or CPU
macOS (Apple Silicon M1/M2/M3/M4)	ARM64	`Llama Desktop Launcher-0.4.1-arm64.dmg`	Metal
macOS (Intel)	x64	`Llama Desktop Launcher-0.4.1.dmg`	Metal

Required disk space depends on the model. Allow room for the GGUF, cache, and engine. Large models can require substantially more RAM or VRAM than the file size alone suggests. Without a compatible GPU, inference may run on the CPU and be slow. The official Linux engine package is built on Ubuntu; other distributions may require compatible system libraries. GitHub Actions compiles native official packages for Windows x64, Linux x64/arm64, and macOS Intel/Apple Silicon.

3. Install the application

The source code is in the [project repository](#). The direct links for version 0.4.1 in the table above will work only after the installers and integrity checksums (SHA256SUMS.txt) are published to the corresponding [GitHub Release](#). To install, download the package matching your platform from the Releases page or build it from source following the instructions in the README.

Windows

1. Open [Llama Desktop Launcher Setup 0.4.1.exe](#).
2. Choose the application installation folder. This is a per-user installer and does not require using [Program Files](#).
3. Select whether to create a desktop shortcut.
4. Finish installation and open the app from the Start menu or desktop shortcut.

Linux

- **DEB:** install with your Debian/Ubuntu compatible package manager.
- **RPM:** install with your Fedora/RHEL compatible package manager.
- **ApplImage:** mark the file executable and run it. ApplImage is portable and remains wherever you place it; a package manager does not install it.

DEB and RPM use the standard locations chosen by the distribution and package manager. Model folders are selected inside the application.

When a package for your platform is available, use the corresponding DEB, RPM, or ApplImage. Experimental portable archives from earlier versions do not replace those packages.

macOS

1. Open the DMG.
2. Drag the app to [Applications](#), or another folder where you want to keep it.
3. Open the app from Finder.

Unsigned development builds may require an explicit exception in macOS security settings. Do not disable system protections globally.

The downloaded engine is extracted into a temporary folder beside its final engine directory. This supports Linux systems where `/tmp` and the user home are separate filesystems; setting `TMPDIR` manually is unnecessary. If installation fails, the launcher removes that staging folder and leaves the configured engine unchanged. The active engine path appears in Settings.

4. First use: set up llama.cpp

On first launch, **Settings** opens automatically if the engine is not ready, there are no models, or no selected model has been saved. After preparing the engine and selecting a model, later launches open **Llama**. You can return to **Settings** from the navigation at any time. The screen explains the remaining setup steps, which you can complete later.

In **Motor llama.cpp**, choose one of these options:

1. **Baixar motor oficial / Download official engine:** choose **Backend do pacote oficial / Official package backend** before clicking the download button. The app downloads release [b11193](#) directly from [ggml-org/llama.cpp](#) on GitHub for the selected platform and backend. On Windows/Linux, the choices depend on the architecture and include Vulkan, NVIDIA CUDA, CPU, and on x64 AMD ROCm and Intel SYCL. macOS uses Metal. CUDA also downloads its matching

official runtime libraries. The engine is stored in your user data folder, outside the app installation folder.

2. **Escolher pasta existente / Choose existing folder:** select a folder containing `llama-server` (or `llama-server.exe`). This lets you reuse an existing installation, including the `b11193-vulkan` directory used by the earlier Windows launcher.

Downloading requires internet access. Choose according to your GPU, independent of CPU brand: Vulkan supports compatible AMD, Intel, and NVIDIA GPUs; CUDA is for NVIDIA, ROCm for AMD, and SYCL for Intel. For an RTX 4060 on Windows x64, start with **NVIDIA CUDA 12.4** and keep the NVIDIA driver current. ROCm and SYCL may require additional runtimes and drivers. If the release has no package for your CPU architecture, select a compatible engine you built or installed yourself. The app checks SHA-256 when the GitHub release API provides a digest. You can replace a selected engine later by choosing a different existing folder or downloading the official package again.

If you used the previous Windows launcher, first select the folder containing your GGUF files. Under **Configurações > Perfis do launcher anterior / Settings > Previous launcher profiles**, click **Importar perfis TSV / Import TSV profiles** and select `launcher-profiles.tsv` from the old launcher folder. Import matches the model's full path; if you moved the GGUF files, select the original folder or adjust profiles manually. Existing profiles in this app are preserved. The result reports imported, missing, already existing, and invalid profiles.

5. Model folders and GGUF files

The **Settings** screen supports **two model locations**. The first defaults to `~/Models/GGUF`; click **Alterar / Change** to select a different folder or drive. The second location is optional and can be added or removed. You can change both later. The **Models** screen scans subfolders for `.gguf` files. Choose the model you want to use there; the app saves the selection and settings associated with that file path.

Even before you download a model, **Models** shows estimates based on available RAM, CPU, detectable GPU/VRAM, and free space in the model folders. These ranges are conservative starting points, not guarantees of compatibility or performance. Always check the GGUF file size, license, author requirements, and available disk space. For MoE models, consider the total weight size, not just the active parameters.

To get a model:

1. In **Models**, explore the options to open Hugging Face in your browser: **Explorar todos os GGUF** to browse all GGUF models unrestricted; **Modelos Quantizados** (where Q4_K_M is suggested as a balanced starting point, with Q5 and Q8 also available); **Modelos MoE** for sparse expert architectures; or **Modelos Densos (sem quantização)** for original unquantized F16/BF16 weights. You can also directly access [all GGUF models](#), [quantized models](#), [MoE models](#), and [dense F16 models](#).
2. Choose a repository. Review the **model license**, RAM/VRAM requirements, quantization, and the author's instructions.
3. Download a `.gguf` file into one of your configured model folders. This version opens Hugging Face in your browser; your browser handles the file download.
4. Return to the app and refresh the model list. In **Models**, review the hardware recommendation and select the GGUF you want.

For a multimodal model, keep exactly one `mmproj*.gguf` file beside the model; the app will pair it automatically. For split GGUF models, keep all shards in the same folder. The list displays only the first shard.

Note: GGUF is a file format, not a guarantee that every model works with this llama.cpp version. Some repositories may require a newer engine.

6. Llama screen: start a conversation

1. Choose a model detected in your configured folders from **Modelo GGUF** ("GGUF model"). If no selected model has been saved yet, the app opens **Configurações** ("Settings"); first download or locate a model from **Modelos** ("Models").

2. To chat inside the app, click **Conversar aqui** (“Chat here”), enter a message, and send it. Enter sends; Shift+Enter inserts a line break. Chat history is not saved.
3. Click **Iniciar servidor** (Start server) to load `llama-server` without opening a page, or **Abrir no navegador** (Open in browser) to start it and open the web interface. When the server is already running, the latter opens the existing interface. The screen shows the access address and configured sessions.
4. When finished, click **Parar servidor** (“Stop server”) in the app.

Use **Abrir CLI no terminal** (“Open CLI in terminal”) to launch `llama-cli` (`llama-cli.exe` on Windows) in a separate terminal with the selected model and current settings. Chat and exit from that terminal. Stop the server or integrated chat before starting the external CLI.

Large models may take several minutes to load. Expand the engine log near the bottom of the window to inspect messages or errors. Closing the browser tab does not stop the server. Use **Stop server** or quit the app.

Start the server without opening a browser

Click **Iniciar servidor** (Start server) to load the model and display the API endpoint without opening a browser. Once ready, **Abrir no navegador** (Open in browser) opens the existing server interface without reloading the model. Use **Parar servidor** (Stop server) to stop it. Stop integrated chat or external CLI before starting a server.

For OpenCode on another computer or VM, select a LAN or Tailscale address before starting. Copy the API link ending in `/v1` and the API key; clients must send the key as a Bearer token. From the client, query `/v1/models`: the model ID is always `modelo-local`, regardless of the selected GGUF. Configure this ID once in OpenCode, then set `model` to `llama-local/modelo-local` to make it the default. Stop the server, select another GGUF and start it again to switch models; existing conversations are not reset automatically. Endpoint and API key must still match the running server. Context limits and tool support depend on the selected model and server settings. In OpenCode, configure an OpenAI-compatible provider with that endpoint and select the model. Project files are edited on the machine running OpenCode, while inference runs on the server. The CLI conversation alone does not provide file-editing tools. On Windows, the external CLI opens in a separate visible PowerShell window.

Use **Copiar comando Bash** (Linux/macOS) or **Copiar comando PowerShell** (Windows) next to the endpoint. Paste into a terminal in the project folder. The command supplies the provider and model for that run through `OPENCODE_CONFIG_CONTENT`, without editing your configuration file. LAN/Tailscale access prompts for the API key with hidden input; the copied command contains no key. OpenCode must be installed. Copy again if the endpoint, context or parallel sessions change. Other OpenCode settings remain effective, except provider/model values overridden by the temporary configuration.

Share access over LAN or Tailscale

With a LAN/Tailscale server running, **Copiar link com acesso** copies an authenticated browser link. **Copiar Bash com acesso** and **Copiar PowerShell com acesso** start OpenCode in the client's project folder without prompting for a key. These commands include a temporary sharing credential; recipients can use the model.

Links and shared commands last only for the current server run. **Revogar acesso compartilhado** invalidates previous links and commands and disconnects shared clients; copy new ones to grant access again. Stopping the server or closing the launcher ends shared access. Restarting generates a new credential. This does not revoke the permanent API key or clients using it directly. The browser removes the token from the address bar after authentication and stores it in an HttpOnly cookie.

Sharing uses an additional port in 8181-8280; the original API uses 8080-8180. The Windows firewall button allows both active ports, limited to the corresponding executables and selected client range. On Linux/macOS, allow both ports in your firewall. OpenCode must be installed on the client. Prefer Tailscale; plain HTTP on LAN does not encrypt content. Copy commands again after changing the model, context, address or parallel sessions. Shared access grants model

usage, not automatic access to server files.

Connect other applications

Under **Conectar aplicativo**, choose OpenCode, Pi, Hermes, Aider, Continue, Cline, Codex or Generic. OpenCode and Aider provide Bash/PowerShell commands to run in your project directory, with the client already installed. Other applications provide **Copiar configuração** and instructions for manually merging it. The launcher does not overwrite client configuration or install clients.

Pi: merge into `~/.pi/agent/models.json`, then select `llama-local/modelo-local` in `/model`. Hermes: merge the `model` block into `config.yaml`, then run `hermes chat`. Current documentation requires at least 64000 tokens per session for tools; the export uses the real context without artificially increasing it. Continue: merge the `models` block into the existing YAML configuration. Cline: select OpenAI Compatible and fill Base URL, API Key and Model ID. Generic exports these fields for other compatible clients.

Copiar configuração com acesso includes the revocable session credential and sharing endpoint. Direct network configuration contains `SUBSTITUA_PELA_CHAVE_DA_API`: replace this with the permanent API key before use. Preserve existing providers, models and settings. Saved shared configurations must be refreshed after revocation or server restart.

Codex is experimental: its configuration requires Responses API on the server or an adapter and has not been validated with this engine. OpenAI compatibility does not guarantee every protocol. Tool support, context and execution quality depend on the GGUF model; vision support is not assumed. The client works on files where it runs.

Integrate application APIs: step by step

Choose **Genérico - OpenAI Compatible** under **Conectar aplicativo** for your own application, service, automation or client supporting OpenAI Chat Completions. The launcher provides the base URL, credential, model ID and limits; it does not expose a public internet API.

1. Choose **Somente este computador** for a client on the server PC, LAN for clients on the same network, or Tailscale for authorized tailnet devices. On a different PC, `127.0.0.1` points to that client itself.
2. Click **Iniciar servidor** and wait for the endpoint. Allow LAN/Tailscale access through your firewall; the Windows button creates rules for the direct API and sharing port in the selected range.
3. Select Generic and choose a mode below. Use the displayed active endpoint: 8080 and 8181 are examples, not fixed ports. Direct API uses 8080-8180; sharing uses 8181-8280.
4. Set Base URL ending in `/v1`, API Key to the matching credential, and Model ID to `modelo-local`. Do not append `/chat/completions` to Base URL if the client already appends it.
5. Test `GET /v1/models`, then `POST /v1/chat/completions` with JSON and `Authorization: Bearer KEY`. The examples already include `/v1` in their base URL, so only append `/models` or `/chat/completions`.

Temporary access or persistent credential

Mode	Where to obtain it	Credential lifetime
Shared	Copiar configuração com acesso ; sharing endpoint and session token	Until revocation, server stop or launcher exit. Copy a fresh configuration after restarting.
Direct over network	Copiar configuração , replacing SUBSTITUA_PELA_CHAVE_DA_API with Copiar chave ; direct API	Preserved across restarts until Nova chave rotates it.
Direct local	Somente este computador , direct endpoint and dummy key local for clients requiring a key field	The local API does not require authentication; this address cannot serve other PCs.

For a persistent integration, use the **direct API** and permanent key. Never mix a permanent key with the sharing port or a temporary token with the direct port. Store the key in the client's secret storage or process environment, outside source code and repositories. Copying direct configuration does not automatically copy the key.

A persistent key does not keep the server running: the launcher and server must remain active. After a PC restart, open the launcher and start the server again. Check the current IP and port and update Base URL if they change. To rotate the key, stop the server, choose **Nova chave**, restart and update direct clients. **Revogar acesso compartilhado** does not revoke the permanent key.

Test on Linux/macOS with Bash and curl

Paste the Base URL and matching credential. The key prompt hides input; for direct local access, enter **local**.

```
(
  read -rp 'Base URL (terminada em /v1): ' LLAMA_BASE_URL
  read -rsp 'API Key: ' LLAMA_API_KEY
  printf '\n'
  curl --fail-with-body -sS \
    -H "Authorization: Bearer $LLAMA_API_KEY" \
    "${LLAMA_BASE_URL%}/models"
  printf '\n'
  curl --fail-with-body -sS \
    -H "Authorization: Bearer $LLAMA_API_KEY" \
    -H 'Content-Type: application/json' \
    "${LLAMA_BASE_URL%}/chat/completions" \
    --data '{"model":"modelo-local","messages":[{"role":"user",
    "content":"Responda apenas: conectado"}],"max_tokens":64,
    "stream":false}'
)
```

The first response should list **modelo-local**. The second answer is in **choices[0].message.content**. These examples request a complete response without streaming. Clients using **stream: true** must process Server-Sent Events (SSE).

Test on Windows with PowerShell

No SDK installation is required. Paste the Base URL and enter the matching key when prompted.

```
& {
  $baseUrl = (Read-Host 'Base URL (terminada em /v1)').TrimEnd('/')
  $secureKey = Read-Host 'API Key' -AsSecureString
  $apiKey = [System.Net.NetworkCredential]::new('', $secureKey).Password
  $headers = @{ Authorization = "Bearer $apiKey" }
  Invoke-RestMethod -Uri "$baseUrl/models" -Headers $headers
  $body = @{
    model = 'modelo-local'
    messages = @( @{ role = 'user'; content = 'Responda: conectado' } )
  }
```

```

    max_tokens = 64
    stream = $false
} | ConvertTo-Json -Depth 5
$response = Invoke-RestMethod -Method Post `
    -Uri "$baseUrl/chat/completions" -Headers $headers `
    -ContentType 'application/json; charset=utf-8' `
    -Body ([System.Text.Encoding]::UTF8.GetBytes($body))
$response.choices[0].message.content
}

```

Python example for your application

Requires Python 3 and uses only its standard library. Set `LLAMA_BASE_URL` and `LLAMA_API_KEY` in the process environment using the chosen URL and credential before running. Do not put the key in URL parameters.

```

import json
import os
from urllib.request import Request, urlopen

base_url = os.environ['LLAMA_BASE_URL'].rstrip('/')
api_key = os.environ['LLAMA_API_KEY']
payload = {
    'model': 'modelo-local',
    'messages': [{'role': 'user', 'content': 'Responda: conectado'}],
    'max_tokens': 64,
    'stream': False,
}
request = Request(
    base_url + '/chat/completions',
    data=json.dumps(payload).encode('utf-8'),
    headers={
        'Authorization': 'Bearer ' + api_key,
        'Content-Type': 'application/json',
    },
    method='POST',
)
with urlopen(request, timeout=120) as response:
    result = json.load(response)
print(result['choices'][0]['message']['content'])

```

The 120-second timeout is an example; larger models may need more time. Available context is the configured server context divided by parallel sessions. Prompt, history, tool definitions and response must all fit; the training context is not the session limit. API responses do not execute code or edit files by themselves: the client application handles those actions.

Troubleshoot application APIs

Result	What to check
Connection refused or timeout	Server started, model loaded, current IP/port, same network or active Tailscale, and firewall.
401 or 403	Bearer header, credential matching the selected mode, and a sharing token that is still valid.
404	Base URL ends in <code>/v1</code> , with no duplicate <code>/v1/v1</code> , and the client uses Chat Completions. Responses API has not been validated with this engine.
400 or context error	Valid JSON, model ID <code>modelo-local</code> , accepted request fields and per-session context limit.
Client does not recognize a model change	Start the new GGUF, check <code>/models</code> , start a new conversation and refresh limits, endpoint or credential as needed.

Tool calling requires a compatible GGUF and chat template; some engines/models require a suitable Jinja template. Do not enable vision, tools or other protocols merely because the endpoint advertises OpenAI compatibility.

7. Advanced settings

Network links and simultaneous sessions

Under **Acesso e sessões / Access and sessions**, select **Somente este computador / This computer only** or a detected LAN/Tailscale address. Choose **1 to 8 simultaneous sessions**. This sets `--parallel` on `llama-server`; sessions share the available context and memory, so start with 1 or 2 for a large model. This setting controls concurrency, while **Context / Contexto** (`-c`) controls the token context size.

For LAN/Tailscale, use **Mostrar chave / Show key**, **Copiar chave / Copy key**, or **Nova chave / New key**. Stop the server before rotating the key. The app opens the local browser interface through an authenticated proxy. **Copiar link da API / Copy API link** gives you the `/v1` endpoint. **Copiar link da interface / Copy interface link** gives you the network web address. Remote API clients must send `Authorization: Bearer YOUR_KEY`; the link alone does not grant access. A remote web client must also support this authentication. Never add the key to the URL or publish it in an open message.

The app does not change firewall rules when starting the server. On Windows, while a LAN/Tailscale server is running, enter the **allowed client address or CIDR range** in **Firewall do Windows / Windows Firewall** and click **Criar regra para a porta ativa / Create rule for active port**. Confirm the Windows administrator prompt. Each rule is limited to the matching executable (`llama-server.exe` for the direct API and the launcher for sharing), its active port and the entered client range. Check the range before confirming; if the port changes, create another rule. On Linux/macOS, use your distribution's firewall controls. The server selects a free port from 8080 to 8180. Prefer Tailscale or a trusted LAN; direct HTTP on a LAN does not encrypt traffic.

Integrated chat and external CLI

If you prefer the original program, click **Abrir CLI no terminal** ("Open CLI in terminal"). The app opens `llama-cli` (`llama-cli.exe` on Windows) in a separate terminal with the selected model and current settings. Use that terminal to chat and close it; **Encerrar conversa** ("End chat") controls only the integrated chat. Stop the server or integrated chat before opening the external CLI. Linux needs a graphical terminal installed. The external terminal may remain open after the launcher closes.

Advanced settings are in the **Ajustes avançados deste modelo** (“Advanced settings for this model”) accordion on the **Llama** screen; it is collapsed initially. Open it to review values, apply an initial suggestion, or compare microbatch. Click **Salvar ajustes deste modelo** (“Save settings for this model”) to save the selected GGUF profile. The **Settings** screen also provides the model/profile selector and keeps its settings open for editing. Settings are saved per model path.

Aplicar sugestão inicial (“Apply initial suggestion”) fills in fields using RAM, CPU thread count, and GGUF information. The estimate may include GPU/VRAM when the system can detect them. It is a starting point, not a guarantee of speed or exact memory use. Review the values and save the profile when you want to keep them.

Field	llama.cpp flag	Meaning
GPU layers	<code>-ngl</code>	<code>auto</code> lets the engine fit layers; <code>all</code> attempts full GPU offload; <code>0</code> selects CPU.
Device	<code>-dev</code>	Empty means automatic selection. Use an ID such as <code>Vulkan0</code> when several GPUs are present.
Context	<code>-c</code>	Number of tokens retained in a conversation. Higher values use more memory.
MoE layers on CPU	<code>-ncmoe</code>	Keeps expert weights for the first N layers in system RAM. Use only with MoE models; empty disables this setting.
K / V cache	<code>-ctk / -ctv</code>	<code>f16</code> favors fidelity; <code>q8_0</code> uses less memory with a small tradeoff. Test other formats per model.
Flash Attention	<code>-fa</code>	<code>auto</code> , <code>on</code> , or <code>off</code> ; availability depends on backend and model.
CPU threads	<code>-t</code>	Threads used by CPU portions. Empty uses the engine default.
Batch	<code>-b</code>	Logical prompt processing batch size.
Microbatch	<code>-ub</code>	Physical chunk size. It cannot be larger than batch.

The app also passes `-fit on` so the engine can adapt to available memory. Leave `-ncmoe` empty for dense models. For an MoE too large for VRAM, adjust CPU MoE layers while watching RAM use and throughput.

8. Review recommendations and benchmark performance

On the **Models** screen, the app prepares a prompt with available hardware information, the selected model, and its settings. Click **Copiar prompt de análise** (“Copy analysis prompt”) to copy it for use elsewhere, or **Analisar com modelo selecionado** (“Analyze with selected model”) to send it through the integrated chat to the current model. If you do not have a model yet, you can copy the prompt and use the local recommendations; AI analysis becomes available after selecting a GGUF.

The model's response is a recommendation for you to review. It does not automatically change or save settings. Check compatibility, memory, license, and actual measurements before applying changes. Local suggestions are also estimates; the benchmark below measures performance on your computer.

To measure microbatch after selecting and loading a GGUF:

1. On the **Llama** screen, open **Ajustes avançados deste modelo** ("Advanced settings for this model"). To get suggestions for several fields, click **Aplicar sugestão inicial** ("Apply initial suggestion"); this does not run the model.
2. Stop the server or active chat and click **Comparar microbatch** ("Compare microbatch"). **llama-bench** tests the current value and another value of up to 128, capped by batch. If they match, it runs only one test.
3. Wait for prompt processing and generation results in tokens per second. The app fills in **Microbatch** with the better value and leaves the other fields unchanged.
4. Review the fields and click **Salvar ajustes deste modelo** ("Save settings for this model") to keep the result without starting the model.

The benchmark may load the model more than once and can take several minutes. It does not save the profile by itself or optimize context, cache, GPU layers, or MoE layers. You can adjust those fields manually.

9. Troubleshooting

Problem	What to check
No models listed	Confirm the model folders and .gguf extension; click Refresh list .
Engine download fails	Check internet access, free space, and GitHub access; retry or select an existing engine folder.
"Package without llama-server"	The release layout may have changed; select a compatible runtime manually.
Server exits during startup	Read the engine log; lower context/batch, try <code>-ngl auto</code> , or reduce GPU layers.
GPU not used	Check the selected backend, GPU compatibility, and drivers; on macOS check that you installed a Metal build. Read the engine log.
Benchmark fails	Ensure the runtime folder contains llama-bench ; try more conservative settings.
Model fails to load	Check all shards, b11193 compatibility, model repository instructions, and available memory.

10. Data, updates, and safe uninstalling

Preferences and profiles are stored as [settings.json](#) in the app's user data folder. The downloaded engine is stored under [engines/](#) in the same area. Model files stay in the folders you chose. The app does not allow a model folder inside the application installation or its internal user data directory.

Before uninstalling, click **Parar servidor / Stop server** if needed, then close the app normally. On Windows, the uninstaller checks whether the app is still open and asks you to close it; click **Retry** after closing. It does not force the app to quit. Use **Installed apps** in Windows Settings. By default, it removes the app and shortcuts while **preserving models, profiles, and the downloaded engine**. Do not use advanced app-data removal options without inspecting the data folder first.

On Linux, remove DEB/RPM with your package manager, or delete the `Applmage` file. On macOS, move the `.app` to Trash. User data and GGUF files remain for manual review. If you want to remove them, do so separately after checking the path and backing up [settings.json](#) if you want to keep profiles. Never delete a model folder simply because it was

selected in the app.

11. Credits

llama.cpp was started by [Georgi Gerganov](#) and is maintained by the [ggml-org contributors](#) under the [MIT license](#). This independent launcher was developed for Célio with Codex, based on the earlier WinForms launcher, and is released under the **MIT license**, with copyright held by Célio. Installer assets must be published separately to the corresponding Release before direct downloads work. The llama.cpp authors do not participate in or endorse this app. Each model has its own license.